Introduction

In the realm of system administration and automation, shell scripting stands as a powerful tool that bridges the gap between users and the operating system. A shell script is a computer program designed to be run by the Unix/Linux shell—a command-line interpreter. It provides a convenient and efficient method for executing sequences of commands, managing files, automating repetitive tasks, and simplifying complex workflows. This assignment aims to explore the fundamental concepts and practical applications of shell scripting. By writing and analyzing various scripts, students gain hands-on experience with essential commands, control structures (such as loops and conditionals), variables, input/output handling, and process management. This practical exposure is not only vital for understanding how operating systems work internally but also for acquiring skills that are highly valued in system administration, DevOps, and software development environments.

The scripting environment in Unix/Linux allows users to develop custom solutions to real-world problems using relatively simple syntax and minimal computational overhead. Whether it is automating backups, monitoring system performance, managing users, or deploying applications, shell scripts play a crucial role. Furthermore, the knowledge of scripting cultivates problem-solving abilities and encourages logical thinking, both of which are essential traits for computer science professionals.

Throughout this assignment, several key topics are covered, including writing basic shell scripts, using conditional and loop constructs, manipulating strings and files, handling errors, and using functions. These exercises are designed to reinforce theoretical understanding through practical implementation. Additionally, this assignment provides a strong foundation for further study in areas like Bash scripting, cron job automation, scripting for security auditing, and integration with tools like Git, Docker, and Ansible.

In conclusion, this assignment is not merely a task—it is an opportunity to gain valuable skills that are directly applicable in the real world. By mastering shell scripting, students become capable of automating mundane tasks, improving system efficiency, and developing robust command-line tools, thereby becoming more proficient and resourceful technologists.

Problem Statement-1:-

Write a shell script to print the prime numbers in a given range. The range is to be supplied as command line arguments.

❒ Objective:-

1. Accept a range of numbers (start and end) as command line arguments to the shell script.
2. Identify and check each number within the specified range to determine if it is a prime number.
3. Print all prime numbers found within the given range to the terminal.

❒ Algorithm:-

1. Start
2. Read command-line arguments into start and end
3. For each number i from start to end - 1:
   (a) If $i < 2$, skip (as numbers < 2 are not prime)
   (b) Set $flag = 1$ (assume i is prime)
   (c) For each number j from 2 to sqrt(i):
      i. If $i\%j == 0$:
         A. Set $flag = 0$ (not prime)
         B. Break
   (d) If $flag == 1$, print i
4. End

❒ Code:-

```
start=$1
end=$2
for ((i=start; i<end; i++)); do
        if [ $i -lt 2 ]; then
                continue
        fi
        flag=1
        for ((j=2; j*j<=i;j++)); do
                if [ $((i%j)) -eq 0 ]; then
                        flag=0
                        break
                fi
        done
        if [ $flag -eq 1 ]; then
                echo $i
        fi
done
```

❒ Output:-

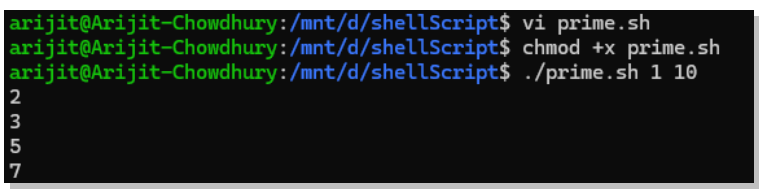


Figure 1: Prime Number print from range by command line argument.

❒ Conclusion:-

The shell script successfully identifies and prints all prime numbers within a user-defined range using command-line arguments. It demonstrates the use of conditional statements, loops, and modular arithmetic in shell scripting. This task not only reinforces fundamental programming concepts but also enhances the ability to write efficient and reusable scripts for numerical problems in Unix/Linux environments.

Problem Statement-2:-

Write a shell script to print the following pattern:

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

❒ Objective:-

The objective of this assignment is to write a shell script that generates and displays Pascal's Triangle up to a specified number of rows. This script demonstrates the use of loops, conditional statements, and arithmetic operations in shell programming. Pascal's Triangle is a mathematical structure used to represent binomial coefficients, and this exercise helps in understanding both combinatorics and basic shell scripting concepts.

❒ Algorithm:-

1. Accept the number of rows as a command-line argument.
2. Initialize the binomial coefficient variable: $coef = 1$.
3. Loop from $i = 0$ to $row - 1$:
   - Print appropriate spaces to center the row.
   - Loop through each element of the row (`j`= 0 to i):
     - If $j == 0$ or $i == 0$, set $coef = 1$.
     - Else, calculate using recurrence:

$$C(i, j) = C(i, j - 1) \times \frac{(i - j + 1)}{j}$$

     - Print the coefficient with formatting.
4. Move to the next line after each row.

❒ Code:-

```
row=$1
coef=1
for ((i=0; i<row; i++)); do
    for ((space=0; space<row-i; space++)); do
        printf " "
    done
    for ((j=0; j<=i; j++));do
        if [ $j -eq 0 ] || [ $i -eq 0 ]; then
            coef=1
        else
            coef=$((coef * (i - j + 1) / j))
        fi
        printf "%2d" $coef
    done
    printf "\n"
done
```

❒ Output:-

Figure 1: Pascal's Triangle.

❒ Conclusion:-

This shell script efficiently prints Pascal's Triangle using loops and arithmetic expressions. By implementing the recurrence relation for binomial coefficients, it avoids the complexity of calculating factorials and enhances performance. The assignment demonstrates key programming concepts like nested loops, conditional statements, and mathematical logic in shell scripting, reinforcing the understanding of combinatorics and algorithmic thinking.

Problem Statement-3:-

Write a shell script which deletes all lines containing the word college from the file supplied as command line argument.

❒ Objective:-

To practice file manipulation and command-line arguments in shell scripting by writing a script that removes all lines containing a specific keyword from a given file. This exercise helps in understanding:

- How to pass and use command-line arguments in shell scripts.
- How to use *grep*, *sed*, or *awk* for pattern matching and text processing.
- Basic file handling operations using shell scripting.

❒ Algorithm:-

1. Start
2. Read the filename from the command-line argument
3. Check if the filename is given
   (a) If not given, display usage message and exit
4. Check if the file exists
   (a) If not found, display error message and exit
5. Use a command to filter out lines containing the word `college`
6. Save the filtered content back to the same file
7. Display confirmation message
8. End

❒ Code:-

```
if [ $# -eq 0 ]; then
    echo "Usage: $0 filename"
    exit 1
fi
if [ ! -f "$1" ]; then
    echo "File not found!"
    exit 2
fi
sed -i '/college/d' "$1"
echo "Lines containing 'college' have been deleted from $1"
```

❒ Output:-

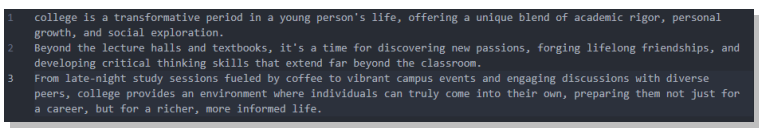


Figure 1.1: Text file containing college word.

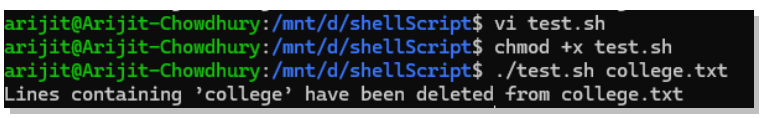


Figure 1.2: Code Execute.

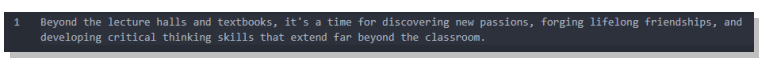


Figure 1.3: Deleted lines.

❒ Conclusion:-

This shell script checks for a valid filename argument and ensures the file exists. It then deletes all lines containing the word `college` from the file using `sed`, and confirms the deletion with a success message.

Problem Statement-4:-

Write a shell script which prints the contents of a text file from line number m to line number n, m and n are supplied from command line argument.

❒ Objective:-

To develop a shell script that extracts and displays a specific range of lines from a text file using command-line arguments. This exercise helps in understanding:

- Command-line argument handling in shell scripting.
- Usage of line processing tools like *sed*.
- File reading and conditional execution in bash scripts.

❒ Algorithm:-

1. Start
2. Read command-line arguments: filename, m, and n
3. Check if all three arguments are provided
   (a) If not, display usage message and exit
4. Check if the file exists
   (a) If not, display error message and exit
5. Use a text-processing command to print lines from m to n
6. Display the output
7. End

❒ Code:-

```
if [ $# -ne 3 ]; then
    echo "Usage: $0 filename m n"
    exit 1
fi
filename=$1
m=$2
n=$3
if [ ! -f "$filename" ]; then
    echo "File '$filename' not found!"
    exit 2
fi
if ! [[ "$m" =~ ^[0-9]+$ && "$n" =~ ^[0-9]+$ && "$m" -le "$n" ]];
then
    echo "Error: m and n should be positive integers with m <= n"
    exit 3
fi
sed -n "${m},${n}p" "$filename"
```

❒ Output:-

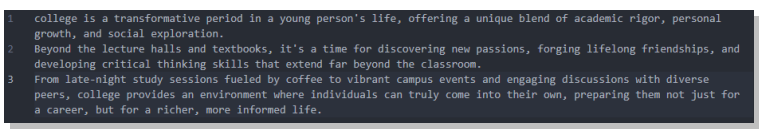


Figure 1.1: text file.

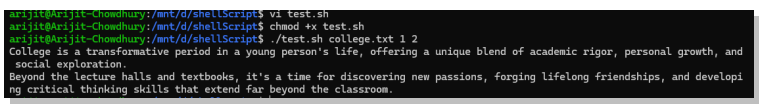


Figure 1.2: printed lines.

❒ Conclusion:-

This shell script accepts a filename and two line numbers `m` and `n` as arguments. It verifies that the file exists and that the line numbers are valid positive integers with `m` less than or equal to `n`. It then prints the lines from `m` to `n` from the specified file using the `sed` command.

Problem Statement-5:-

Write a shell script to display all the lines of a file, starting with a vowel. Provide the file name as the command line argument.

❒ Objective:-

To create a shell script that filters and displays only those lines from a file that begin with a vowel. The filename is passed as a command-line argument. This enhances understanding of:

- Command-line argument usage.
- Regular expressions in text processing.
- File filtering using *grep* in shell scripting.

❒ Algorithm:-

1. Start
2. Read the filename from the command-line argument
3. Check if the filename is provided
   (a) If not, display usage message and exit
4. Check if the file exists
   (a) If not, display error message and exit
5. Use a pattern-matching command to find lines starting with a vowel (A, E, I, O, U or a, e, i, o, u)
6. Display the matching lines
7. End

❒ Code:-

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 filename"
    exit 1
fi
filename=$1
if [ ! -f "$filename" ]; then
    echo "Error: File '$filename' not found!"
    exit 2
fi
grep -i '^[aeiou]' "$filename"
```

❒ Output:-

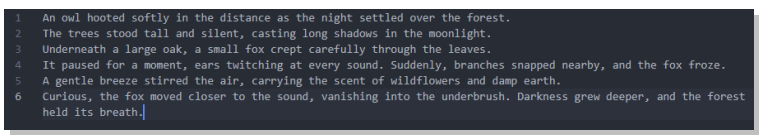


Figure 1.1: Text file.

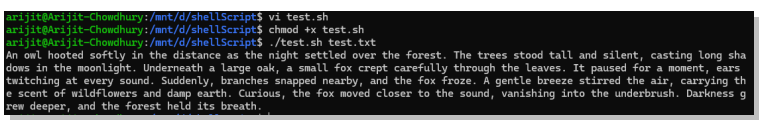


Figure 1.2: Vowel line print.

❒ Conclusion:-

This shell script checks if exactly one filename is provided as an argument and verifies that the file exists. It then uses the `grep` command to display all lines from the file that start with a vowel (case-insensitive).

Problem Statement-6:-

Write a shell script that receives any number of filename as any nu me as arguments and check if every argument supplied as a file or a directory and report accordingly. Whenever the arguments are in a file, the number of lines of it is also reported.

❒ Objective:-

To write a shell script that processes multiple command-line arguments, identifies whether each argument is a file or directory, and provides detailed information including the number of lines if it is a file. This task helps in learning:

- Handling multiple command-line arguments in shell scripts.
- Using conditional statements to distinguish between files and directories.
- Counting lines in a file using command-line utilities.

❒ Algorithm:-

1. Start
2. Read all command-line arguments
3. If no arguments are given:
   (a) Display usage message and exit
4. For each argument:
   (a) If it is a regular file:
      i. Display that it is a file
      ii. Count and display the number of lines in it
   (b) Else if it is a directory:
      i. Display that it is a directory

(c) Else:

i. Display that it is neither a file nor a directory

5. End

❒ Code:-

```
if [ $# -eq 0 ]; then
    echo "Usage: $0 file_or_directory1 [file_or_directory2 ...]"
    exit 1
fi
for item in "$@"
do
    if [ -f "$item" ]; then
        echo "'$item' is a regular file."
        line_count=$(wc -l < "$item")
        echo "Number of lines in '$item': $line_count"
    elif [ -d "$item" ]; then
        echo "'$item' is a directory."
    else
        echo "'$item' is neither a file nor a directory."
    fi
    echo "----------------------------"
done
```

❒ Output:-

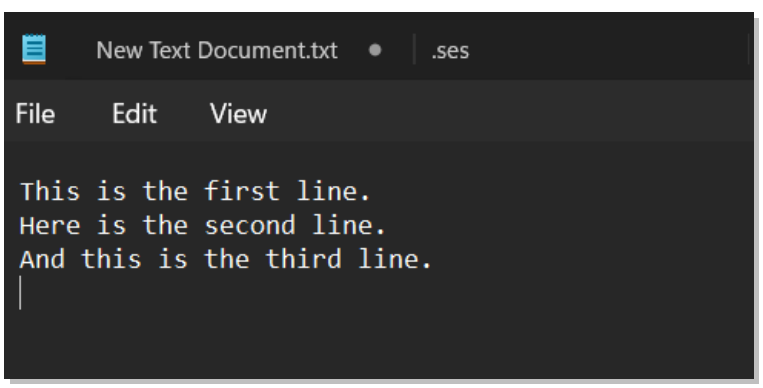


Figure 1.1: Text file.

❒ Conclusion:-

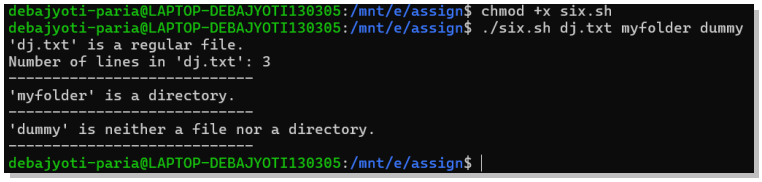


Figure 1.2: Checking.

This shell script processes each argument passed to it, checks whether it is a regular file or a directory, and displays appropriate information. For regular files, it also counts and prints the number of lines using the `wc -l` command. If the item is neither a file nor a directory, it prints a corresponding message.

Problem Statement-7:-

Write a shell script that takes a name of a folder as a command line argument, and produce a file that contains the names of all sub folders with size 0 (that is empty sub folders).

❒ Objective:-

To develop a shell script that accepts a folder name as a command-line argument and identifies all empty subfolders within it, saving their names to an output file. This exercise aims to strengthen understanding of:

- Directory traversal using shell scripting.
- Detecting empty directories.
- Command-line argument handling and file output operations.

❒ Algorithm:-

1. Start
2. Read the folder name from the command-line argument
3. If no argument is given:
   (a) Display usage message and exit
4. Check if the argument is a valid directory
   (a) If not, display error and exit
5. Create or clear an output file named `empty_folders.txt`
6. Traverse the folder recursively
7. For each subfolder:
   (a) If it is empty, add its name to `empty_folders.txt`
8. Display a message with the output file name

9. End

❒ Code:-

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 folder_name"
    exit 1
fi
folder=$1
if [ ! -d "$folder" ]; then
    echo "Error: '$folder' is not a directory!"
    exit 2
fi
echo "Empty subfolders listed in 'empty_subfolders.txt'"
```

❒ Output:-

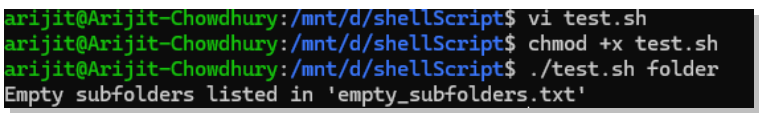


Figure 1.1: execution.

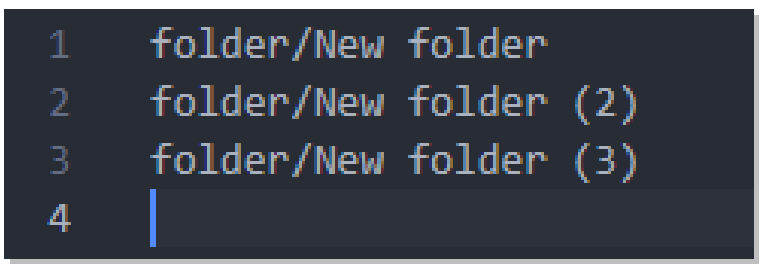


Figure 1.2: listed empty folders.

❒ Conclusion:-

This shell script checks if exactly one directory name is provided and verifies whether it exists. If valid, it proceeds to identify empty subfolders within the given directory and indicates that their names are listed in the file `empty_subfolders.txt`.

Problem Statement-8:-

Write a shell script to simulate 'wc' command.

❒ Objective:-

The objective of this assignment is to write a shell script that simulates the functionality of the 'wc' (word count) command, which displays the number of lines, words, and characters in a given text file.

This assignment aims to:

- Develop the ability to work with file input and text processing in shell scripting.
- Reinforce the use of commands like 'wc', 'cat', 'tr', and 'wc -l', 'wc -w', 'wc -c'.
- Practice the implementation of command substitution and argument validation.
- Strengthen scripting logic to create alternatives to standard Linux utilities.

❒ Algorithm:-

1. Start
2. Read the filename from the command-line argument
3. If no filename is provided:
   (a) Display usage message and exit
4. Check if the file exists
   (a) If not, display error message and exit
5. Count the number of lines in the file
6. Count the number of words in the file

7. Count the number of characters in the file
8. Display the counts in the format: lines words characters filename
9. End

❒ Code:-

```
[ $# -ne 1 ] && echo "Usage: $0 filename" && exit 1
[ ! -f "$1" ] && echo "File not found!" && exit 2
lines=$(wc -l < "$1")
words=$(wc -w < "$1")
chars=$(wc -m < "$1")
echo "$lines $words $chars $1"
```

❒ Output:-

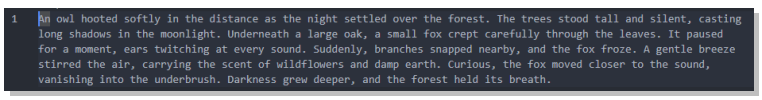


Figure 1.1: paragraph.

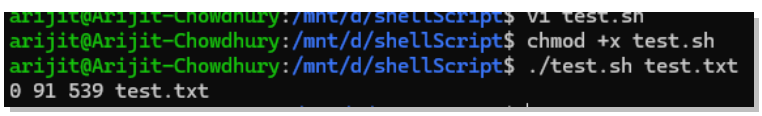


Figure 1.2: word count.

❒ Conclusion:-

This shell script verifies that a filename is provided and the file exists. It then calculates and displays the number of lines, words, and characters in the file using the `wc` command.

Problem Statement-9:-

Write a shell script to perform bubble sort on an array of numbers. If 1 is given in command line argument, array will be sorted in ascending order and if 2 is given, array will be sorted in descending order.

❒ Objective:-

To write a shell script that implements the Bubble Sort algorithm on an array of numbers. The sorting order is determined by a command-line argument: if `1` is provided, the array is sorted in ascending order; if `2` is provided, the array is sorted in descending order. This assignment helps to understand:

- Implementation of sorting algorithms in shell scripts.
- Handling arrays and loops in shell scripting.
- Command-line argument parsing and conditional logic.

❒ Algorithm:-

1. Start
2. Read sorting order from command-line argument
   (a) If 1, sort in ascending order
   (b) If 2, sort in descending order
3. Prompt the user to enter the number of elements
4. Read the array elements
5. Perform bubble sort:
   (a) For each element from 0 to $n - 2$:
      i. Compare adjacent elements
      ii. If out of order (based on selected mode), swap them

6. Display the sorted array
7. End

❒ Code:-

```
read -p "Enter numbers separated by space: " -a arr
n=${#arr[@]}
if [ $# -ne 1 ] || { [ "$1" != "1" ] && [ "$1" != "2" ]; }; then
    echo "Usage: $0 1 (asc) or 2 (desc)"
    exit 1
fi
for ((i = 0; i < n - 1; i++)); do
    for ((j = 0; j < n - i - 1; j++)); do
        if [ "$1" -eq 1 ] && [ "${arr[j]}" -gt "${arr[j+1]}" ];
        then
            temp=${arr[j]}
            arr[j]=${arr[j+1]}
            arr[j+1]=$temp
        elif [ "$1" -eq 2 ] && [ "${arr[j]}" -lt "${arr[j+1]}" ];
        then
            temp=${arr[j]}
            arr[j]=${arr[j+1]}
            arr[j+1]=$temp
        fi
    done
done
echo "Sorted array: ${arr[@]}"
```

❒ Output:-

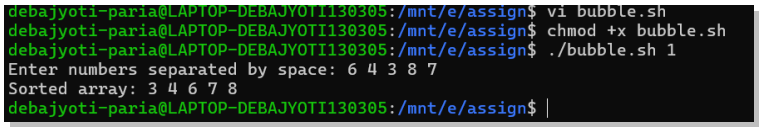


Figure 1.1: Ascending Order.

```
debajyoti-paria@LAPTOP-DEBAJYOTI130305:/mnt/e/assign$ ./bubble.sh 2
Enter numbers separated by space: 6 3 7 1 2
Sorted array: 7 6 3 2 1
debajyoti-paria@LAPTOP-DEBAJYOTI130305:/mnt/e/assign$
```

Figure 1.2: Descending Order.

❒ Conclusion:-

This shell script reads a list of numbers from the user and sorts them using the bubble sort algorithm. It takes one command-line argument to determine the sorting order: `1` for ascending and `2` for descending. The script then displays the sorted array based on the selected order.

Problem Statement-10:-

Write a shell script/shell program to print Fibonacci series. The range input is to be supplied as command line argument.

❒ Objective:-

The objective of this assignment is to write a shell script that prints the Fibonacci series up to a given number, where the range (limit) is provided as a command line argument. The script should validate the input and generate the series correctly up to the specified value. This assignment is designed to:

1. Strengthen skills in shell scripting using command line arguments.
2. Develop logic-building ability by implementing iterative control structures (like while or for loops).
3. Enhance understanding of variable handling, arithmetic operations, and input validation in shell scripts.
4. Practice creating user-interactive scripts that accept dynamic inputs.

❒ Algorithm:-

1. Start
2. Read the range (number of terms) from command-line argument
3. If no argument is given:
   (a) Display usage message and exit
4. Initialize two variables: $a = 0$, $b = 1$
5. Repeat for the given range:

(a) Print a

(b) Calculate next term as $fn = a + b$

(c) Set $a = b$

(d) Set $b = fn$

6. End

❒ Code:-

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 range"
    exit 1
fi
n=$1
a=0
b=1
echo -n "Fibonacci series up to $n: "
while [ $a -le $n ]; do
    echo -n "$a "
    fn=$((a + b))
    a=$b
    b=$fn
done
echo
```

❒ Output:-

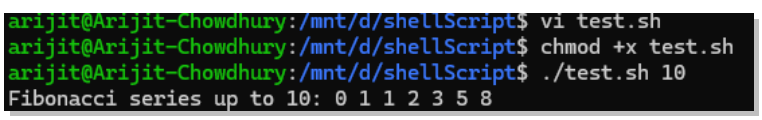


Figure 1: Fibonacci series.

❒ Conclusion:-

This shell script generates and displays the Fibonacci series up to a given range `n`, provided as a command-line argument. It uses a `while` loop to compute the series iteratively and prints each term until the value exceeds `n`.

Problem Statement-11:-

Write a shell script to find the second highest number from a list of numbers without using sorting.

❒ Objective:-

To write a shell script that finds the second highest number from a list of user-provided numbers without using any sorting technique. This assignment helps in understanding:

- Array and variable manipulation in shell scripting.
- Use of conditional logic to track maximum and second maximum values.
- Basic algorithm design without relying on built-in sorting utilities.

❒ Algorithm:-

1. Start
2. Read a list of numbers into an array
3. Initialize two variables:
   - `max` = very small value
   - `second_max` = very small value
4. For each number in the array:
   (a) If number ¿ `max`:
      i. Set `second_max` = `max`
      ii. Set `max` = number
   (b) Else if number ¿ `second_max` and number `max`:
      i. Set `second_max` = number
5. If `second_max` is unchanged, print "Second highest not found"

6. Else, display `second_max`
7. End

❒ Code:-

```
read -p "Enter numbers separated by space: " -a arr
if [ ${#arr[@]} -lt 2 ]; then
    echo "Need at least two numbers"
    exit 1
fi
max=-999999
second=-999999
for num in "${arr[@]}"; do
    if [ "$num" -gt "$max" ]; then
        second=$max
        max=$num
    elif [ "$num" -gt "$second" ] && [ "$num" -ne "$max" ]; then
        second=$num
    fi
done
if [ "$second" -eq -999999 ]; then
    echo "No second highest number found
    (all numbers may be same)"

else
    echo "Second highest number is: $second"
fi
```

❒ Output:-

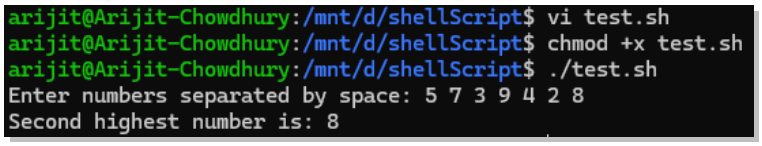


Figure 1: second highest.

❒ **Conclusion:-**

This shell script takes a list of numbers as input from the user and identifies the second highest number. It ensures that at least two distinct numbers are entered and handles cases where all numbers may be the same.

Problem Statement-12:-

Write a shell script to perform binary search on a set of data.

❒ Objective:-

To write a shell script that performs binary search on a set of sorted data elements to find the position of a specified target value. This assignment aims to enhance understanding of:

- Implementing search algorithms in shell scripts.
- Efficiently accessing and manipulating array elements.
- Using control structures like loops and conditional statements for algorithmic logic.

❒ Algorithm:-

1. Start
2. Read a sorted list of numbers into an array
3. Read the target number to search
4. Initialize variables:
   - `low` = 0
   - `high` = length of array - 1
5. Repeat while `low` ≤ `high`:
   (a) Calculate `mid` = (low + high) / 2
   (b) If `arr[mid]` == target:
      i. Display position and exit
   (c) Else if target ¡ `arr[mid]`:
      i. Set `high` = mid - 1
   (d) Else:

i. Set `low` = mid + 1

6. If loop ends and element is not found:
   (a) Display "Element not found"
7. End

❒ Code:-

```
read -p "Enter sorted numbers (space separated): " -a arr
read -p "Enter the number to search: " key
low=0
high=$((${#arr[@]} - 1))
found=0
while [ $low -le $high ]; do
    mid=$(((low + high) / 2))
    if [ "${arr[mid]}" -eq "$key" ]; then
        echo "Number found at index $mid"
        found=1
        break
    elif [ "$key" -lt "${arr[mid]}" ]; then
        high=$((mid - 1))
    else
        low=$((mid + 1))
    fi
done
[ $found -eq 0 ] && echo "Number not found"
```

❒ Output:-

Figure 1: Binary Search.

❒ **Conclusion:-**

This shell script performs a binary search on a sorted list of numbers entered by the user. It efficiently searches for a given number and reports its index if found, or a message indicating it was not found.

Problem Statement-13:-

Write a shell script to prepare word frequency of a given text file.

❒ Objective:-

The objective of this assignment is to write a shell script that analyzes a given text file and prepares a word frequency report, showing how many times each unique word appears in the file.

❒ Algorithm:-

1. Start
2. Read the filename from the command-line argument
3. If the number of arguments is not 1:
   (a) Display usage message and exit
4. Check if the file exists:
   (a) If not, display error message and exit
5. Process the file:
   (a) Replace spaces and punctuation with newlines to extract words
   (b) Convert all characters to lowercase
   (c) Remove empty lines
   (d) Sort the words
   (e) Count the frequency of each unique word
   (f) Sort the result in descending order of frequency
6. Display the result (word frequencies)
7. End

❒ Code:-

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 filename"
    exit 1
fi
file=$1
if [ ! -f "$file" ]; then
    echo "Error: File '$file' not found!"
    exit 2
fi
tr -s '[:space:][:punct:]' '\n' < "$file" | tr 'A-Z' 'a-z' |
grep -v '^$' | sort | uniq -c | sort -nr
```

❒ Output:-

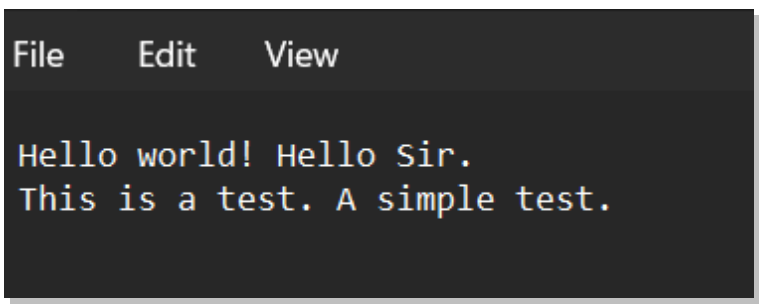


Figure 1.1: Text File.

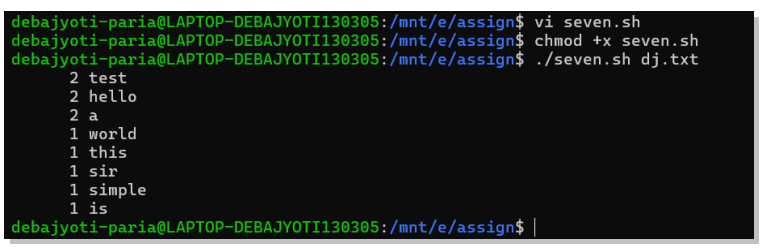


Figure 1.2: Count.

❒ **Conclusion:-**

This shell script reads a text file and generates a sorted list of unique words along with their frequencies, in descending order. It normalizes the text by converting it to lowercase, replaces punctuation and spaces with newlines, removes empty lines, and uses `sort`, `uniq -c`, and `sort -nr` to compute and display the word frequency.

Problem Statement-14:-

Write a shell script that will take an input file and remove identical lines (or duplicate lines from the file).

❒ Objective:-

To write a shell script that accepts a text file as input and removes all duplicate (identical) lines, retaining only the unique lines in the output.

This assignment helps in understanding:

- File handling and reading in shell scripts.
- Use of text processing tools like `sort` and `uniq`.
- Practical applications of command-line utilities for data cleaning.

❒ Algorithm:-

1. Start
2. Read the filename from the command-line argument
3. If the number of arguments is not equal to 1:
   (a) Display usage message and exit
4. Check if the file exists:
   (a) If not, display an error message and exit
5. Sort the contents of the file
6. Remove duplicate lines using a uniqueness filter
7. Save the result in a file named `unique_lines.txt`
8. Display a message confirming the output
9. End

❒ Code:-

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 input_file"
    exit 1
fi
file=$1

if [ ! -f "$file" ]; then
    echo "Error: File '$file' not found!"
    exit 2
fi
sort "$file" | uniq > unique_lines.txt
echo "Duplicate lines removed. Output written to
'unique.txt'"
```

❒ Output:-

Figure 1.1: Printed Output.

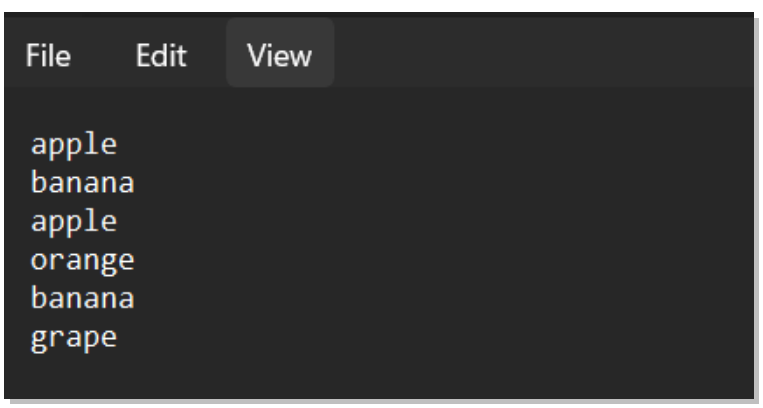


Figure 1.2: Inputed Text File.

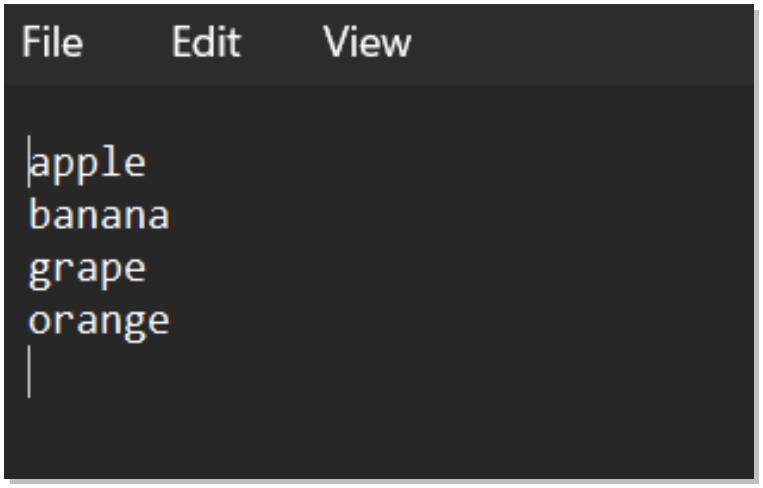


Figure 1.3: Auto Created Text File Removed Duplicates.

❒ **Conclusion:-**

This shell script accepts a filename as a command-line argument, verifies its existence, and removes duplicate lines by sorting the file and using the `uniq` command. The resulting unique lines are saved to `unique_lines.txt`.

Problem Statement-15:-

Write a program that transfers data from one file to a new file excluding vowels.

❒ Objective:-

The objective of this assignment is to write a shell script that reads data from a source file and creates a new file by copying the content while excluding all vowels (both uppercase and lowercase).

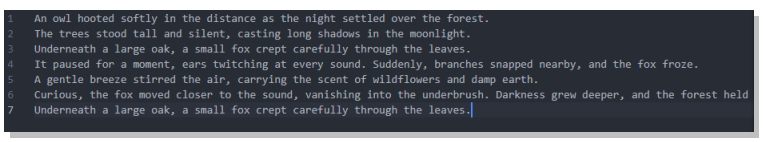
❒ Algorithm:-

1. Start
2. Read two command-line arguments: `input_file` and `output_file`
3. If the number of arguments is not equal to 2:
   (a) Display usage message and exit
4. Check if `input_file` exists:
   (a) If not, display an error message and exit
5. Read the contents of `input_file`
6. Remove all vowels (both uppercase and lowercase) from the content
7. Write the result to `output_file`
8. Display confirmation message
9. End

❒ Code:-

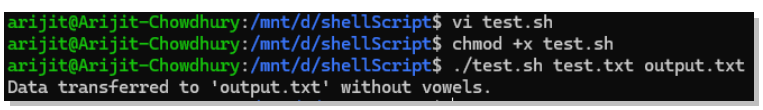
```
if [ $# -ne 2 ]; then
    echo "Usage: $0 input_file output_file"
    exit 1
fi
input=$1
output=$2

if [ ! -f "$input" ]; then
    echo "Error: File '$input' not found!"
    exit 2
fi
tr -d 'aeiouAEIOU' < "$input" > "$output"
echo "Data transferred to '$output' without vowels."
```

❒ Output:-



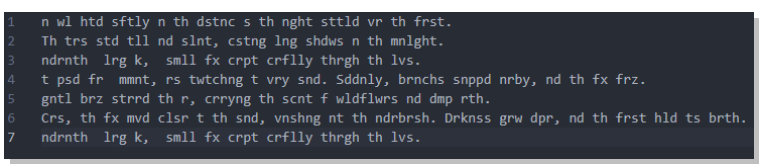
```
An owl hooted softly in the distance as the night settled over the forest.
The trees stood tall and silent, casting long shadows in the moonlight.
Underneath a large oak, a small fox crept carefully through the leaves.
It paused for a moment, ears twitching at every sound. Suddenly, branches snapped nearby, and the fox froze.
A gentle breeze stirred the air, carrying the scent of wildflowers and damp earth.
Curious, the fox moved closer to the sound, vanishing into the underbrush. Darkness grew deeper, and the forest held
Underneath a large oak, a small fox crept carefully through the leaves.
```

Figure 1.1: text file.



```
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ vi test.sh
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ chmod +x test.sh
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ ./test.sh test.txt output.txt
Data transferred to 'output.txt' without vowels.
```

Figure 1.2: file transferred.



```
 n wl htd sftly n th dstnc s th nght sttld vr th frst.
 Th trs std tll nd slnt, cstng lng shdws n th mnlght.
 ndrnth  lrg k,  smll fx crpt crflly thrgh th lvs.
 t psd fr  mmnt, rs twtchng t vry snd. Sddnly, brnchs snppd nrby, nd th fx frz.
 gntl brz strrd th r, crryng th scnt f wldflwrs nd dmp rth.
 Crs, th fx mvd clsr t th snd, vnshng nt th ndrbrsh. Drknss grw dpr, nd th frst hld ts brth.
 ndrnth  lrg k,  smll fx crpt crflly thrgh th lvs.
```

Figure 1.3: ouput.txt.

❒ **Conclusion:-**

This shell script takes an input and an output file as arguments, checks the existence of the input file, and removes all vowels (both uppercase and lowercase) from its content using the `tr` command. The result is saved to the specified output file.

Problem Statement-16:-

Write a shell script to find the roots of a quadratic equation $ax^2 + bx + c = 0$, where $a \neq 0$. Consider all the cases.

❒ Objective:-

To write a shell script that calculates the roots of a quadratic equation of the form $ax^2 + bx + c = 0$, where $a \neq 0$. The script should handle all possible cases: real and distinct roots, real and equal roots, and complex (imaginary) roots.

This assignment aims to develop:

- Mathematical computation using shell scripting.
- Conditional branching to handle different types of roots based on the discriminant.
- Command-line input handling and basic floating-point arithmetic.

❒ Algorithm:-

1. Start
2. Prompt user to input coefficients a, b, and c
3. Read the values of a, b, and c
4. If $a = 0$:
   (a) Display error: "Not a quadratic equation" and exit
5. Calculate discriminant $d = b^2 - 4ac$
6. Check the value of discriminant d
   (a) If $d > 0$:
      i. Display: "Two real and distinct roots"

ii. Calculate roots using:

$$\text{root}_1 = \frac{-b + \sqrt{d}}{2a}, \quad \text{root}_2 = \frac{-b - \sqrt{d}}{2a}$$

(b) Else if $d = 0$:

i. Display: "One real and repeated root"

ii. Calculate root using:

$$\text{root} = \frac{-b}{2a}$$

(c) Else ($d < 0$):

i. Display: "Complex roots"

ii. Calculate:

$$\text{real part} = \frac{-b}{2a}, \quad \text{imaginary part} = \frac{\sqrt{-d}}{2a}$$

iii. Display complex roots as:

$$\text{real} \pm \text{imag}i$$

7. End

❒ **<u>Code:-</u>**

```
echo "Enter coefficients a, b, and c (for ax^2 + bx + c = 0):"
read -p "a: " a
read -p "b: " b
read -p "c: " c
if [ "$a" -eq 0 ]; then
    echo "This is not a quadratic equation (a cannot be 0)"
    exit 1
fi
d=$(echo "$b * $b - 4 * $a * $c" | bc)
if [ "$d" -gt 0 ]; then
    echo "Discriminant is $d > 0 → Two real and distinct roots."
    root1=$(echo "scale=4; (-$b + sqrt($d)) / (2 * $a)" | bc -l)
```

```
    root2=$(echo "scale=4; (-$b - sqrt($d)) / (2 * $a)" | bc -l)
    echo "Roots: $root1 and $root2"
elif [ "$d" -eq 0 ]; then
    echo "Discriminant is $d = 0 → One real and repeated root."
    root=$(echo "scale=4; -$b / (2 * $a)" | bc -l)
    echo "Root: $root"
else
    echo "Discriminant is $d < 0 → Complex roots."
    real=$(echo "scale=4; -$b / (2 * $a)" | bc -l)
    imag=$(echo "scale=4; sqrt(-$d) / (2 * $a)" | bc -l)
    echo "Roots: $real + ${imag}i and $real - ${imag}i"
fi
```

❒ Output:-

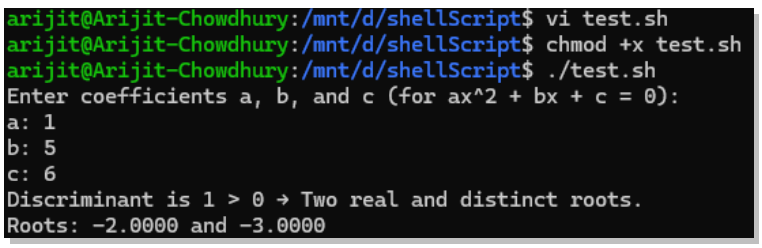


Figure 1: Quadratic equation solve.

❒ Conclusion:-

This shell script solves a quadratic equation of the form $ax^2+bx+c=0$. It first checks whether the coefficient a is non-zero, then calculates the discriminant to determine the nature of the roots. Based on the discriminant, it computes and displays either two real and distinct roots, one real repeated root, or two complex roots.

Problem Statement-17:-

Write a shell script to perform selection sort on an array of numbers in descending order.

❒ Objective:-

To write a shell script that performs Selection Sort on an array of numbers to arrange them in descending order.

This assignment helps in understanding:

- Implementation of sorting algorithms using shell scripting.
- Array manipulation and indexing in bash.
- Use of loops and conditional statements to perform comparisons and swaps.

❒ Algorithm:-

1. Start
2. Read an array of numbers from the user
3. Determine the size of the array (n)
4. Repeat for $i = 0$ to $n - 2$:
   (a) Set `max_idx` $= i$
   (b) Repeat for $j = i + 1$ to $n - 1$:
      i. If `arr[j] > arr[max_idx]`, then set `max_idx` $= j$
   (c) Swap `arr[i]` and `arr[max_idx]`
5. Display the sorted array in descending order
6. End

❒ Code:-

```
read -p "Enter numbers separated by space: " -a arr
n=${#arr[@]}
for ((i = 0; i < n - 1; i++)); do
    max_idx=$i
    for ((j = i + 1; j < n; j++)); do
        if [ "${arr[j]}" -gt "${arr[max_idx]}" ]; then
            max_idx=$j
        fi
    done
    temp=${arr[i]}
    arr[i]=${arr[max_idx]}
    arr[max_idx]=$temp
done
echo "Sorted array in descending order: ${arr[@]}"
```

❒ Output:-

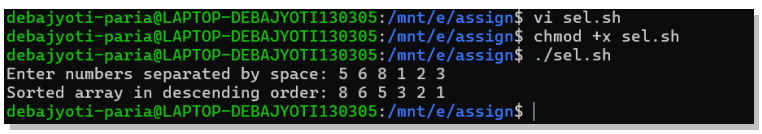


Figure 1: Sorted Array In Descending Order.

❒ Conclusion:-

This shell script sorts a list of user-entered numbers in descending order using the selection sort algorithm. It repeatedly selects the maximum element from the unsorted part and places it at the beginning. The sorted array is then displayed.

Problem Statement-18:-

Write a shell script to perform the GCD of two integers.

❒ Objective:-

The objective of this assignment is to write a shell script that calculates the Greatest Common Divisor (GCD) of two integers provided by the user (via input or command line arguments).

❒ Algorithm:-

1. Start
2. Read two numbers: a and b
3. Remove any negative sign from both a and b
4. While $b \neq 0$, repeat:
   (a) Set `temp` $= b$
   (b) Set $b = a \mod b$
   (c) Set $a =$ `temp`
5. When loop ends, a is the GCD
6. Print "GCD is: a"
7. End

❒ Code:-

```
read -p "Enter first number: " a
read -p "Enter second number: " b
a=${a#-}
b=${b#-}
while [ "$b" -ne 0 ]; do
    temp=$b
    b=$((a % b))
    a=$temp
```

```
done
echo "GCD is: $a"
```

❒ Output:-

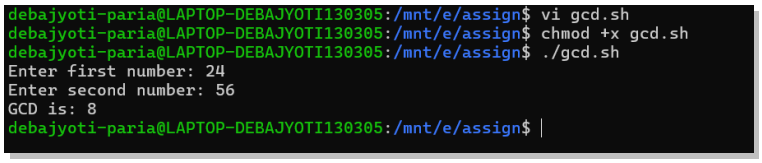


Figure 1: Gcd Of 24 and 56.

❒ Conclusion:-

This shell script calculates the Greatest Common Divisor (GCD) of two user-provided numbers using the Euclidean algorithm. It also handles negative inputs by converting them to their absolute values before computation.

Problem Statement-19:-

Write a shell script to implement a calculator performing addition, subtraction, multiplication and division operations using menu driven concept. The choice of operation must be taken through command line argument.

❒ Objective:-

To write a shell script that implements a basic calculator using a menu-driven approach. The script should perform addition, subtraction, multiplication, and division based on the user's choice, which is provided as a command-line argument.

This assignment helps in understanding:

- Command-line argument parsing in shell scripts.
- Implementation of arithmetic operations using shell scripting.
- Use of conditional statements and control structures to build menu-driven programs.

❒ Algorithm:-

1. Start
2. Check if the number of command-line arguments is not equal to 1
   - If true, display usage message and exit
3. Read two numbers: a and b from user
4. Based on the command-line argument, perform:
   - Case 1: Addition
     - Compute and display $a + b$
   - Case 2: Subtraction

    - Compute and display $a - b$
  - Case 3: Multiplication
    - Compute and display $a \times b$
  - Case 4: Division
    - If $b = 0$, display "Error: Division by zero"
    - Else compute and display $a \div b$
  - Default: Display "Invalid choice"
5. End

❒ **<u>Code:-</u>**

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 {1|2|3|4}"
    echo "1. Addition"
    echo "2. Subtraction"
    echo "3. Multiplication"
    echo "4. Division"
    exit 1
fi
read -p "Enter first number: " a
read -p "Enter second number: " b
case $1 in
    1)
        echo "Result: $((a + b))"
        ;;
    2)
        echo "Result: $((a - b))"
        ;;
    3)
        echo "Result: $((a * b))"
        ;;
    4)
        if [ "$b" -eq 0 ]; then
            echo "Error: Division by zero!"
        else
```

```
            echo "Result: $((a / b))"
        fi
        ;;
    *)
        echo "Invalid choice. Use: 1 (Add), 2 (Sub),
        3 (Mul), 4 (Div)"
        ;;
esac
```

❒ Output:-

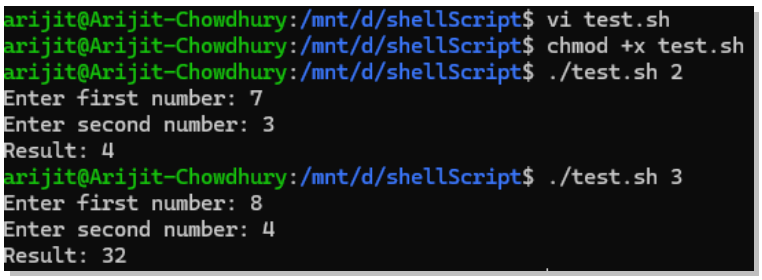


Figure 1: Calculator.

❒ Conclusion:-

This shell script performs basic arithmetic operations—addition, subtraction, multiplication, and division—based on the user's choice provided as a command-line argument. It reads two numbers from the user and uses a `case` statement to execute the selected operation, with proper handling for division by zero and invalid input.

Problem Statement-20:-

Write a shell program to check whether the given number and its reverse are same. Take the input through command line argument.

❒ Objective:-

The objective of this assignment is to write a shell script that checks whether a given number and its reverse are the same (i.e., whether the number is a palindrome), where the number is passed as a command line argument. This assignment is intended to:

- Strengthen the understanding of command line argument handling in shell scripts.
- Practice control flow statements and loops for number manipulation.
- Enhance logic-building skills by implementing digit extraction and reversal.
- Develop familiarity with conditional checking and basic input validation.
- Encourage writing user-friendly and functional shell scripts for numeric operations.

❒ Algorithm:-

1. **Start**
2. Check if the number of command-line arguments is not equal to 1
   - If true, display usage message: `Usage: $0 number`
   - Exit the script
3. Assign the first command-line argument to variable `num`

4. Reverse the contents of **num** using **rev** command and store in variable **rev**
5. Compare **num** and **rev**
   - If equal, print: **<num> is a palindrome**
   - Else, print: **<num> is not a palindrome**
6. **End**

❒ Code:-

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 number"
    exit 1
fi
num=$1
rev=$(echo "$num" | rev)

if [ "$num" -eq "$rev" ]; then
    echo "$num is a palindrome."
else
    echo "$num is not a palindrome."
fi
```

❒ Output:-

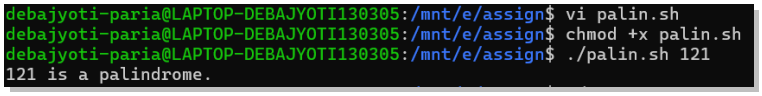


Figure 1.1: Check For Palindrome Number.

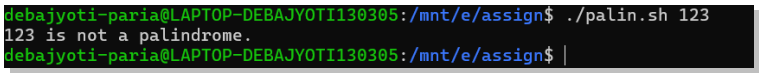


Figure 1.2: Check For Non Palindrome Number.

❒ Conclusion:-

This shell script checks whether a given number is a palindrome. It reverses the input number and compares it with the original. If both are equal, the number is a palindrome; otherwise, it is not.

Problem Statement-21:-

Write a shell script to convert the file content from lower case to upper case or upper case to lower case using menu driven concept.

❒ Objective:-

To write a shell script that converts the content of a file from lowercase to uppercase or from uppercase to lowercase using a menu-driven approach.

This assignment helps in understanding:

- File reading and writing in shell scripting.
- Text transformation using commands like `tr`.
- Implementation of menu-driven logic using control structures.
- Command-line interaction and user input handling.

❒ Algorithm:-

1. **Start**
2. Check if the number of command-line arguments is not equal to 1
   - If true, display usage message: `Usage: $0 filename`
   - Exit the script
3. Assign the first argument to variable `file`
4. Check if the file exists
   - If not, display error message and exit
5. Display options to the user:
   - 1. Convert to UPPER case
   - 2. Convert to lower case

6. Read user's choice into variable `choice`
7. Use `case` to perform corresponding action:
   - Case 1: Use `tr 'a-z' 'A-Z'` to convert file content to uppercase
   - Case 2: Use `tr 'A-Z' 'a-z'` to convert file content to lowercase
   - Default: Display `Invalid choice`
8. **End**

❒ **Code:-**

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 filename"
    exit 1
fi
file=$1
if [ ! -f "$file" ]; then
    echo "Error: File '$file' not found!"
    exit 2
fi
echo "Choose an option:"
echo "1. Convert to UPPER case"
echo "2. Convert to lower case"
read -p "Enter your choice (1 or 2): " choice
case $choice in
    1)
        tr 'a-z' 'A-Z' < "$file"
        ;;
    2)
        tr 'A-Z' 'a-z' < "$file"
        ;;
    *)
        echo "Invalid choice."
        ;;
esac
```

❒ Output:-

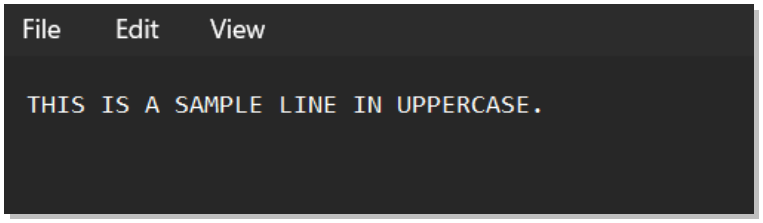


Figure 1.1: Uppercase File.

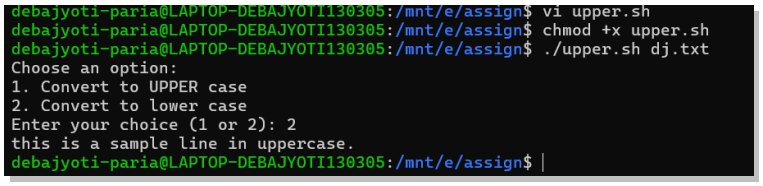


Figure 1.2: Converted to Lowercase.

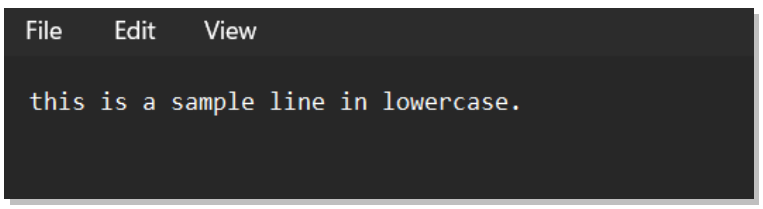


Figure 1.3: Lowercase File.

```
debajyoti-paria@LAPTOP-DEBAJYOTI130305:/mnt/e/assign$ chmod +x upper.sh
debajyoti-paria@LAPTOP-DEBAJYOTI130305:/mnt/e/assign$ ./upper.sh dj.txt
Choose an option:
1. Convert to UPPER case
2. Convert to lower case
Enter your choice (1 or 2): 1
THIS IS A SAMPLE LINE IN LOWERCASE.
debajyoti-paria@LAPTOP-DEBAJYOTI130305:/mnt/e/assign$
```

Figure 1.4: Converted to Uppercase.

❒ Conclusion:-

This shell script allows the user to convert the contents of a given file to either uppercase or lowercase. It verifies the file's existence and uses the `tr` command based on the user's choice to perform the case conversion.

Problem Statement-22:-

Write a shell script to implement bubble sort method.

❒ Objective:-

To write a shell script that implements the Bubble Sort algorithm to sort an array of numbers. This assignment helps in understanding:

- The logic and working of the Bubble Sort algorithm.
- Array handling and manipulation in shell scripting.
- Use of loops and conditional statements to perform comparisons and swaps.

❒ Algorithm:-

1. **Start**
2. Prompt the user to enter numbers separated by space
3. Read the input numbers into an array `arr`
4. Determine the number of elements `n` in the array
5. Perform Bubble Sort:
   - For each index `i` from 0 to `n - 2`
     - For each index `j` from 0 to `n - i - 2`
       - If `arr[j]` ¿ `arr[j+1]`, then:
       - Swap `arr[j]` and `arr[j+1]`
6. Display the sorted array
7. **End**

❒ Code:-

```
echo "Enter numbers separated by space:"
read -a arr
n=${#arr[@]}
for ((i = 0; i < n-1; i++))
do
    for ((j = 0; j < n-i-1; j++))
    do
        if [ "${arr[j]}" -gt "${arr[$((j+1))]}" ]; then
            # Swap elements
            temp=${arr[j]}
            arr[j]=${arr[$((j+1))]}
            arr[$((j+1))]=$temp
        fi
    done
done
echo "Sorted array:"
echo "${arr[@]}"
```

❒ Output:-

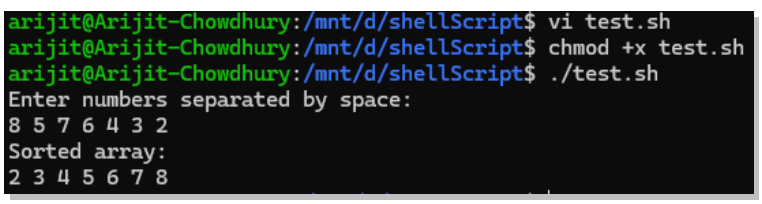


Figure 1: Bubble sort.

❒ Conclusion:-

This shell script sorts a list of numbers entered by the user in ascending order using the bubble sort algorithm. It repeatedly compares and swaps adjacent elements to arrange the array in the correct order, and finally displays the sorted result.

Problem Statement-23:-

Write a shell script to draw the following pattern.

```
*
**
***
****
***
**
*
```

❒ Objective:-

To write a shell script that prints a specific star pattern using nested loops. This assignment helps in understanding:

- Looping constructs (for/while) in shell scripting.
- Printing formatted output to the terminal.
- Logical thinking for constructing symmetrical patterns using loops.

❒ Algorithm:-

1. **Start**
2. Loop from $i = 1$ to 4
   - Loop from $j = 1$ to i
     - Print * without newline
   - Print newline after inner loop
3. Loop from $i = 3$ down to 1
   - Loop from $j = 1$ to i

    - Print * without newline
  - Print newline after inner loop

4. **End**

❒ **Code:-**

```
rows=4
for (( i=1; i<=rows; i++ ))
do
    for (( s=rows-i; s>0; s-- ))
    do
        echo -n " "
    done
    for (( j=1; j<=2*i-1; j++ ))
    do
        echo -n "*"
    done
    echo
done
for (( i=rows-1; i>=1; i-- ))
do
    for (( s=rows-i; s>0; s-- ))
    do
        echo -n " "
    done
    for (( j=1; j<=2*i-1; j++ ))
    do
        echo -n "*"
    done
    echo
done
```

❒ Output:-

```
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ vi test.sh
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ chmod +x test.sh
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ ./test.sh
   *
  ***
 *****
*******
 *****
  ***
   *
```

Figure 1: Star pattern.

❒ Conclusion:-

This shell script prints a diamond-like star pattern using nested `for` loops. It first prints an increasing sequence of stars (from 1 to 4), followed by a decreasing sequence (from 3 to 1), creating a symmetric pattern.

Problem Statement-24:-

Write a shell script to print prime factors of a given number, supplied as command line argument.

❒ Objective:-

To write a shell script that takes a number as a command-line argument and prints all of its prime factors.

This assignment helps in understanding:

- Command-line argument handling in shell scripting.
- Implementation of basic mathematical algorithms (prime factorization).
- Use of loops and conditional statements for number theory operations.

❒ Algorithm:-

1. **Start**
2. Check if exactly one command-line argument is passed
   - If not, display usage message and exit
3. Let `num` = command-line argument
4. Check if `num` is a valid positive integer greater than 1
   - If not, display error and exit
5. Display message: "Prime factors of `num` are:"
6. Initialize `i = 2`
7. While `num >= i`, do:
   - If `num % i == 0`, then:

    - Print `i`
    - Set `num = num / i`
  - Else:
    - Increment `i = i + 1`

8. **End**

❒ Code:-

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 <positive-integer>"
    exit 1
fi
num=$1
if ! [[ "$num" =~ ^[0-9]+$ ]] || [ "$num" -le 1 ]; then
    echo "Error: Please enter an integer greater than 1."
    exit 1
fi
echo "Prime factors of $num are:"
i=2
while [ $num -ge $i ]
do
    if [ $((num % i)) -eq 0 ]; then
        echo $i
        num=$((num / i))
    else
        i=$((i + 1))
    fi
done
```

❒ Output:-

```
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ vi test.sh
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ chmod +x test.sh
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ ./test.sh 7
Prime factors of 7 are:
7
arijit@Arijit-Chowdhury:/mnt/d/shellScript$ ./test.sh 9
Prime factors of 9 are:
3
3
```

Figure 1: Star pattern.

❒ Conclusion:-

This shell script computes and displays the prime factors of a given positive integer greater than 1. It validates the input and uses a loop to divide the number by the smallest possible factor iteratively until all prime factors are found.

Problem Statement-25:-

Write a shell script that takes a word from the user and find out the frequency of the word in a given file.

❒ Objective:-

To write a shell script that takes a word as input from the user and calculates the frequency of that word in a specified file. This assignment helps in understanding:

- User input handling in shell scripts.
- File reading and text processing.
- Usage of command-line tools like `grep`, `wc`, and pattern matching to count word occurrences.

❒ Algorithm:-

1. **Start**
2. Prompt the user to enter a filename and store it in `filename`
3. Check if the file `filename` exists
   - If the file does not exist, display "File not found" and exit
4. Prompt the user to enter the word to search and store it in `word`
5. Use `grep` with `-o` and word boundaries (`\b`) to find all occurrences of `word` in the file
6. Count the number of lines output by `grep` using `wc -l`, and store the result in `count`

7. Display: "The word '`word`' occurred `count` times in the file '`filename`'."

8. **End**

❒ Code:-

```
read -p "Enter the file name: " filename
if [ ! -f "$filename" ]; then
    echo "File '$filename' not found."
    exit 1
fi
read -p "Enter the word to search: " word
count=$(grep -o "\b$word\b" "$filename" | wc -l)
echo "The word '$word' occurred $count times in the file
'$filename'."
```

❒ Output:-

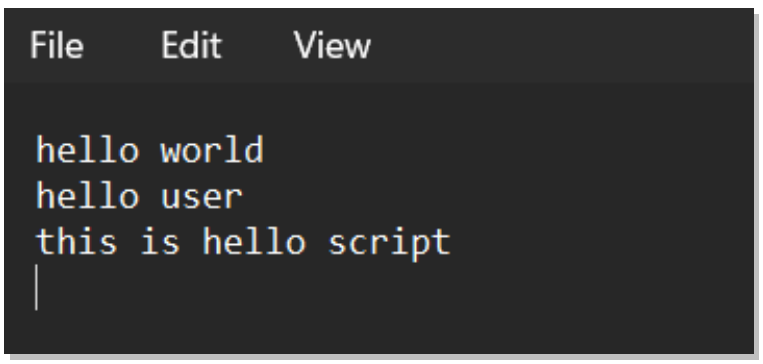


Figure 1.1: Text File.

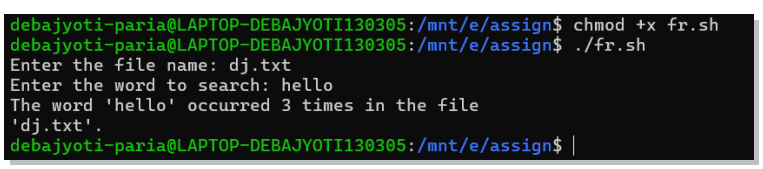


Figure 1.2: Count the Number.

❒ Conclusion:-

This shell script takes a filename and a word as input, checks whether the file exists, and then counts the number of times the

exact word appears in the file using the `grep` and `wc` commands. It ensures that only whole word matches are counted.

Problem Statement-26:-

Write a shell script to check whether a given word is a palindrome or not. The size of the word must be at least of 4 characters.

❒ Objective:-

The objective of this assignment is to write a shell script that checks whether a given word is a palindrome, where the word is provided as a command line argument, and its length must be at least 4 characters.

This assignment aims to: - Reinforce handling of command line arguments and string operations in shell scripting. - Practice input validation based on word length. - Strengthen logic implementation for reversing and comparing strings. - Develop scripts that perform basic text analysis tasks effectively.

❒ Algorithm:-

1. **Start**
2. Prompt the user to enter a word and store it in variable `word`
3. Calculate the length of `word` and store it in `length`
4. If `length < 4`, then:
   - Display "The word must be at least 4 characters long."
   - Exit the program
5. Reverse the word and store it in `reverse`
6. Compare `word` and `reverse`
   - If they are equal, display "The word is a palindrome."

- Else, display "The word is not a palindrome."

7. **End**

❒ Code:-

```
read -p "Enter a word: " word
length=${#word}
if [ $length -lt 4 ]; then
    echo "The word must be at least 4 characters long."
    exit 1
fi
reverse=$(echo "$word" | rev)
if [ "$word" = "$reverse" ]; then
    echo "The word '$word' is a palindrome."
else
    echo "The word '$word' is not a palindrome."
fi
```

❒ Output:-

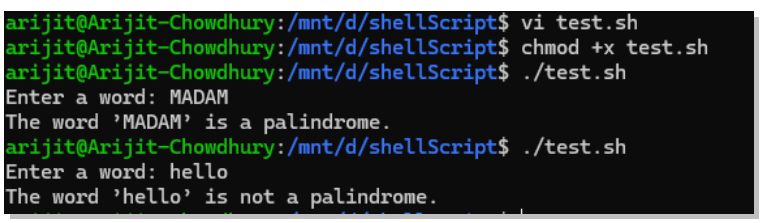


Figure 1: Palindrome.

❒ Conclusion:-

This shell script checks whether a user-entered word (with a minimum length of 4 characters) is a palindrome. It compares the word with its reverse and displays an appropriate message based on the result.

Problem Statement-27:-

Write a shell script to count the words, lines and characters of a given file. File name should be provided at run time.

❒ Objective:-

To write a shell script that takes a filename as input at runtime and counts the total number of words, lines, and characters in the file.

This assignment helps in understanding:

- Command-line input handling in shell scripting.
- File processing using standard Unix commands.
- Utilization of tools like `wc` to analyze text files.

❒ Algorithm:-

1. **Start**
2. Check if exactly one command-line argument is provided
   - If not, display: "Usage: $0 ¡filename¿" and exit
3. Store the argument in variable `filename`
4. Check if the file `filename` exists
   - If not, display: "Error: File 'filename' does not exist." and exit
5. Use `wc -l` to count and store number of lines in `lines`
6. Use `wc -w` to count and store number of words in `words`
7. Use `wc -c` to count and store number of characters in `chars`

8. Display:
   - File name
   - Number of lines
   - Number of words
   - Number of characters
9. **End**

❒ **Code:-**

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi
filename=$1
if [ ! -f "$filename" ]; then
    echo "Error: File '$filename' does not exist."
    exit 1
fi
lines=$(wc -l < "$filename")
words=$(wc -w < "$filename")
chars=$(wc -c < "$filename")
echo "File: $filename"
echo "Lines     : $lines"
echo "Words     : $words"
echo "Characters: $chars"
```

❒ Output:-

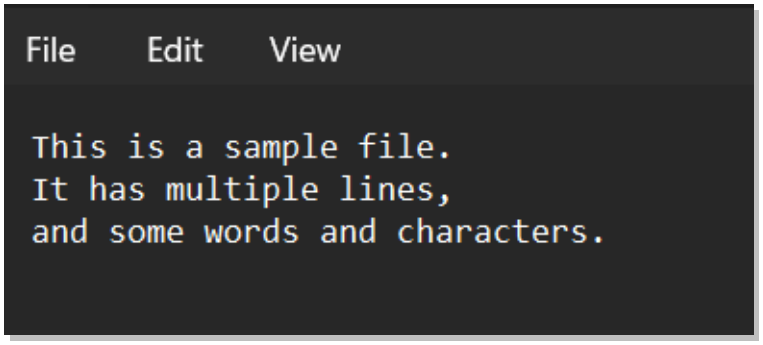


Figure 1.1: Text File.

Figure 1.2: Counted Output.

❒ Conclusion:-

This shell script takes a filename as input, verifies its existence, and then calculates and displays the number of lines, words, and characters in the file using the `wc` command.

Problem Statement-28:-

Write a shell script that gets executed at the moment of user login and it displays Good Morning,Good afternoon, Good Evening, Good Night, depending upon the time at which the user logs on.

❒ Objective:-

The objective of this assignment is to write a shell script that executes automatically at the time of user login and displays an appropriate greeting such as "Good Morning", "Good Afternoon", "Good Evening", or "Good Night" based on the current system time.

This assignment aims to:

- Demonstrate the use of system time and conditional logic in shell scripting.
- Familiarize with customizing user login experience via shell configuration files.
- Enhance understanding of time-based automation using shell scripts.
- Develop practical skills in managing user environment behavior during login.

❒ Algorithm:-

1. **Start**
2. Retrieve the current hour using `date +%H` and store it in variable `hour`
3. Check the value of `hour`:

- If $5 \leq$ `hour` < 12, display "Good Morning!"
- Else if $12 \leq$ `hour` < 17, display "Good Afternoon!"
- Else if $17 \leq$ `hour` < 21, display "Good Evening!"
- Else, display "Good Night!"

4. **End**

❒ Code:-

```
hour=$(date +%H)
if [ $hour -ge 5 ] && [ $hour -lt 12 ]; then
    echo "Good Morning!"
elif [ $hour -ge 12 ] && [ $hour -lt 17 ]; then
    echo "Good Afternoon!"
elif [ $hour -ge 17 ] && [ $hour -lt 21 ]; then
    echo "Good Evening!"
else
    echo "Good Night!"
fi
```

❒ Output:-

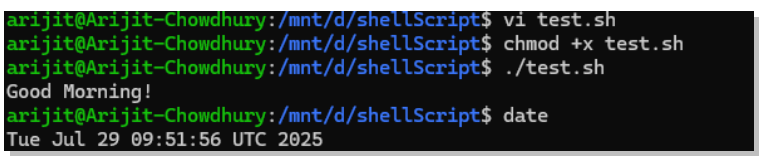


Figure 1: Time checker msg.

❒ Conclusion:-

This shell script retrieves the current hour and displays an appropriate greeting based on the time of day. It categorizes the time into morning, afternoon, evening, or night using conditional checks.

Problem Statement-29:-

Write a shell script to find a number using sequential search method.

❒ Objective:-

To write a shell script that implements the Sequential Search (Linear Search) algorithm to find a number in a list of numbers. This assignment helps in understanding:

- Implementation of searching algorithms in shell scripting.
- Array handling and user input processing.
- Use of loops and conditional statements to search for a specific value.

❒ Algorithm:-

1. **Start**
2. Prompt the user to enter array elements (space-separated) and store them in array `arr`
3. Prompt the user to enter the number to search and store it in variable `key`
4. Initialize a flag variable `found` to 0
5. Loop through the array from index 0 to length - 1
   - If `arr[i] == key`, then:
     - Print the position of the key (i+1)
     - Set `found = 1`
     - Exit the loop

6. After the loop, if `found == 0`, print "Number not found in the array"
7. **End**

❒ Code:-

```
echo "Enter array elements (space-separated):"
read -a arr
read -p "Enter the number to search: " key
found=0
for (( i=0; i<${#arr[@]}; i++ ))
do
    if [ "${arr[$i]}" -eq "$key" ]; then
        echo "Number $key found at position $((i+1))."
        found=1
        break
    fi
done
if [ $found -eq 0 ]; then
    echo "Number $key not found in the array."
fi
```

❒ Output:-

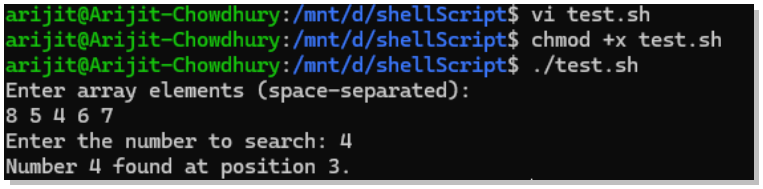


Figure 1: Sequential Search.

❒ Conclusion:-

This shell script performs a linear search on an array of numbers entered by the user. It checks each element to find the specified key and prints its position if found, otherwise it notifies that the number is not present in the array.

Problem Statement-30:-

Write a shell script to find a number using binary search technique.

❒ Objective:-

The objective of this assignment is to write a shell script that implements the binary search algorithm to find a given number in a sorted list of numbers provided by the user.

This assignment aims to:

- Strengthen understanding of fundamental searching algorithms using shell scripting.
- Practice handling arrays, user input, and function implementation in Bash.
- Demonstrate the use of loop structures and conditional statements for logic control.
- Enhance problem-solving skills using algorithmic thinking in shell scripts.

❒ Algorithm:-

1. **Start**
2. Prompt the user to enter sorted array elements and store them in array `arr`
3. Prompt the user to enter the number to search and store in variable `key`
4. Initialize `low = 0`, `high = length of array - 1`, and `found = 0`

5. While `low` is less than or equal to `high`, repeat:
   5.1. Compute `mid = (low + high) / 2`
   5.2. If `arr[mid] == key`:
      – Print position `mid + 1`
      – Set `found = 1`
      – Break the loop
   5.3. Else if `arr[mid] < key`, set `low = mid + 1`
   5.4. Else set `high = mid - 1`
6. If `found == 0`, print "Number not found in the array"
7. **End**

❒ **Code:-**

```
echo "Enter sorted array elements (space-separated):"
read -a arr
read -p "Enter the number to search: " key
low=0
high=$((${#arr[@]} - 1))
found=0
while [ $low -le $high ]
do
    mid=$(( (low + high) / 2 ))
    if [ "${arr[$mid]}" -eq "$key" ]; then
        echo "Number $key found at position $((mid+1))."
        found=1
        break
    elif [ "${arr[$mid]}" -lt "$key" ]; then
        low=$((mid + 1))
    else
        high=$((mid - 1))
    fi
done
if [ $found -eq 0 ]; then
```

```
    echo "Number $key not found in the array."
fi
```

❒ Output:-

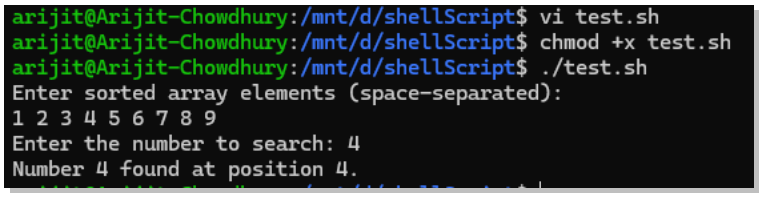


Figure 1: Binary search.

❒ Conclusion:-

This shell script implements the binary search algorithm on a sorted array provided by the user. It efficiently searches for a specified number and reports its position if found; otherwise, it informs the user that the number is not present in the array.

Problem Statement-31:-

Write a shell script to find out the factorial of a given number.

❒ Objective:-

The objective of this assignment is to write a shell script to calculate the factorial of a given non-negative integer supplied through the command line or user input.

This assignment aims to:

1. Practice the use of loops and arithmetic operations in shell scripting.
2. Reinforce the concept of iterative problem-solving using control structures.
3. Demonstrate input validation and error handling in scripts.
4. Enhance logic development skills through mathematical computation in shell.

❒ Algorithm:-

1. **Start**
2. Prompt the user to enter a non-negative integer and store it in variable `num`
3. If the input is not a non-negative integer:
   - Display error message
   - Exit the program
4. Initialize `fact = 1`
5. For `i` from 1 to `num`, repeat:

- Multiply `fact` by `i`
- Update `fact`

6. Display the factorial result `fact`
7. **End**

❒ Code:-

```
read -p "Enter a non-negative integer: " num
if ! [[ "$num" =~ ^[0-9]+$ ]]; then
    echo "Error: Input must be a non-negative integer."
    exit 1
fi
fact=1
for (( i=1; i<=num; i++ ))
do
    fact=$((fact * i))
done
echo "Factorial of $num is: $fact"
```

❒ Output:-

Figure 1: Factorial.

❒ Conclusion:-

This shell script calculates the factorial of a non-negative integer entered by the user. It validates the input and uses a `for` loop to compute the factorial iteratively, then displays the result.

Problem Statement-32:-

Write a shell script to reverse a string and check whether it is a palindrome.

❒ Objective:-

To write a shell script that reverses a given string and checks whether the reversed string is the same as the original, thereby determining if it is a palindrome.

This assignment helps in understanding:

- String manipulation techniques in shell scripting.
- Use of command-line tools like `rev` or manual reversal logic.
- Implementation of conditional checks to compare strings.

❒ Algorithm:-

1. **Start**
2. Prompt the user to enter a string and store it in variable `str`
3. Reverse the string using the `rev` command and store it in `rev_str`
4. Compare `str` with `rev_str`
   - If both are equal, display: "`str` is a palindrome"
   - Else, display: "`str` is not a palindrome"
5. **End**

❒ Code:-

```
read -p "Enter a string: " str
rev_str=$(echo "$str" | rev)
if [ "$str" = "$rev_str" ]; then
    echo "'$str' is a palindrome."
else
    echo "'$str' is not a palindrome."
fi
```

❒ Output:-

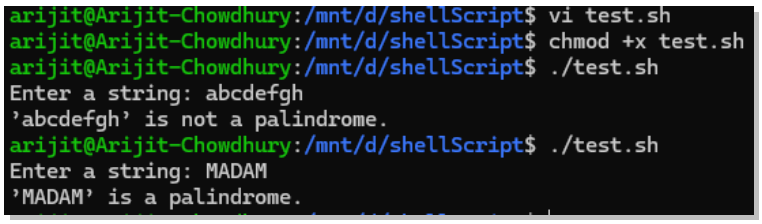


Figure 2: **Figure 1:** Palindrome with rev.

❒ Conclusion:-

This shell script checks whether a given string is a **palindrome**. It uses the `rev` command to reverse the input string and compares the reversed string with the original. If both strings match, it confirms the input is a palindrome; otherwise, it states it is not.

This program demonstrates a simple yet effective use of string manipulation and conditional statements in shell scripting, making it a helpful exercise for beginners learning Bash scripting.

Problem Statement-33:-

Write a shell script for menu-based system to insert records for employees with employee ID, name, designation, salary in a data file, also display records when necessary. Display salary for the employee asked.

❒ Objective:-

The objective of this assignment is to write a menu-driven shell script to manage employee records by allowing insertion of employee details (ID, name, designation, salary) into a data file, displaying all records, and retrieving salary information for a specific employee.

❒ Algorithm:-

1. Start
2. Set `datafile` as `"employee_data.txt"`
3. **If** file `employee_data.txt` does not exist:
   - Create the file using `touch`
4. Repeat the following steps until user selects Exit:
   a. Display the menu:
      - 1. Insert New Employee Record
      - 2. Display All Records
      - 3. Display Salary of an Employee (by ID)
      - 4. Exit

   b. Prompt user to enter choice

   c. **Switch case** on user's choice:

- **Case 1: Insert New Employee**
  - Prompt for Employee ID, Name, Designation, and Salary
  - Append these values as a comma-separated line to `datafile`
- **Case 2: Display All Records**
  - If file is non-empty, display column headers and contents
  - Else, show "No records found"
- **Case 3: Display Salary of an Employee**
  - Prompt for Employee ID
  - Search for matching line in the file
  - If found, extract and display salary field
  - Else, display "Employee not found"
- **Case 4: Exit**
  - Display exit message
  - Terminate the program
- **Default:** Display "Invalid choice"

5. End

❒ **Code:-**

```
datafile="employee_data.txt"
if [ ! -f "$datafile" ]; then
    touch "$datafile"
fi
while true
do
    echo ""
    echo "=========== Employee Record System ==========="
    echo "1. Insert New Employee Record"
```

```
echo "2. Display All Records"
echo "3. Display Salary of an Employee (by ID)"
echo "4. Exit"
echo "==============================================="
read -p "Enter your choice [1-4]: " choice
case $choice in
    1)
        echo "---- Insert New Employee ----"
        read -p "Enter Employee ID: " eid
        read -p "Enter Employee Name: " name
        read -p "Enter Designation: " desig
        read -p "Enter Salary: " salary
        echo "$eid,$name,$desig,$salary" >> $datafile
        echo "Record inserted successfully."
        ;;
    2)
        echo "---- All Employee Records ----"
        if [ -s $datafile ]; then
            echo "EmpID, Name, Designation, Salary"
            cat $datafile
        else
            echo "No records found."
        fi
        ;;
    3)
        echo "---- Search Employee Salary ----"
        read -p "Enter Employee ID: " search_id
        result=$(grep "^$search_id," $datafile)
        if [ -n "$result" ]; then
            salary=$(echo "$result" | cut -d',' -f4)
            echo -e "ID $search_id \n Salary:$salary"
        else
             echo "Employee with ID $search_id not found."
        fi
        ;;
    4)
```

```
            echo "Exiting. Thank you!"
            exit 0
            ;;
        *)
            echo "Invalid choice. Please select between 1 and 4."
            ;;
    esac
done
```

❒ Output:-

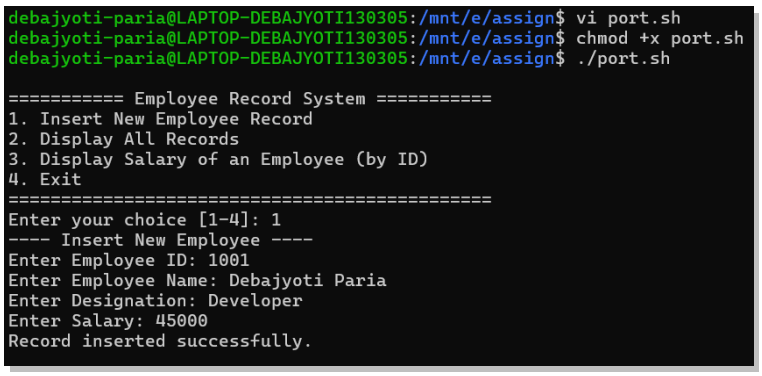


Figure 1.1: Enter New Id.

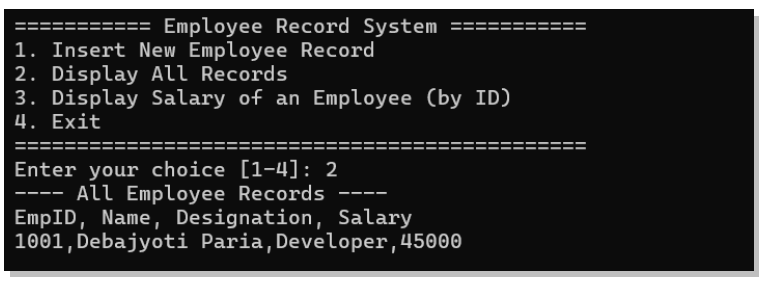


Figure 1.2: Display New Id.

```
=========== Employee Record System ===========
1. Insert New Employee Record
2. Display All Records
3. Display Salary of an Employee (by ID)
4. Exit
=============================================
Enter your choice [1-4]: 1
---- Insert New Employee ----
Enter Employee ID: 1002
Enter Employee Name: Arijit Chowdhury
Enter Designation: Developer
Enter Salary: 50000
Record inserted successfully.
```

Figure 1.3: Enter New Id.

```
=========== Employee Record System ===========
1. Insert New Employee Record
2. Display All Records
3. Display Salary of an Employee (by ID)
4. Exit
=============================================
Enter your choice [1-4]: 3
---- Search Employee Salary ----
Enter Employee ID: 1002
ID 1002
 Salary:50000
```

Figure 1.4: Find Salary by Id.

```
=========== Employee Record System ===========
1. Insert New Employee Record
2. Display All Records
3. Display Salary of an Employee (by ID)
4. Exit
=============================================
Enter your choice [1-4]: 4
Exiting. Thank you!
```

Figure 1.5: Exit With Thanking.

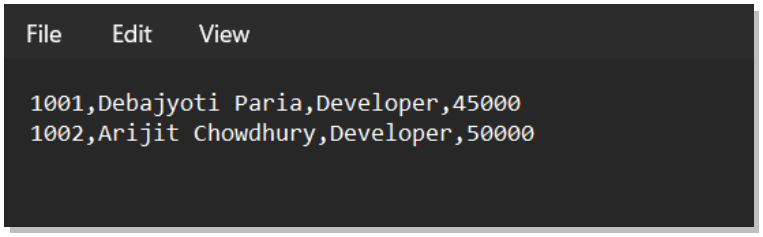


Figure 1.6: Saved File.

❒ Conclusion:-

This shell script implements a basic **Employee Record Management System** using a command-line interface. It allows the user to:

- Insert new employee records into a file (`employee_data.txt`).
- Display all existing employee records.
- Search and display the salary of an employee using their ID.
- Exit the program gracefully.

The script uses standard Bash constructs such as loops, conditionals, and file handling operations. It also employs common utilities like `grep` and `cut` to manipulate and extract data. The system ensures data persistence by storing records in a file and handles invalid inputs effectively.

Overall, this project is a simple yet effective way to understand basic shell scripting, especially for beginners interested in learning file processing and menu-driven programming in Unix/Linux environments.

Acknowledgment

It is with great respect and deep gratitude that I take this opportunity to express my sincere appreciation to Ms. Shrabana Dutta, under whose valuable guidance and constant encouragement I have successfully completed this Shell Script assignment.

Throughout the course of this work, she has been a constant source of inspiration. Her depth of knowledge, structured teaching methodology, and unwavering support played a vital role in shaping my understanding of shell scripting concepts. She provided me with a strong foundation and encouraged me to think critically and apply logic while writing scripts—something that has not only helped me academically but has also broadened my practical thinking.

The discipline and analytical thinking involved in shell scripting were made accessible and enjoyable under her mentorship. I have gained a deeper appreciation for automation and scripting techniques, which I believe will be immensely valuable in my future studies and professional career.

I am also thankful for her emphasis on originality, thoroughness, and attention to detail, which helped me enhance the overall quality of this assignment. Her guidance has not only improved my technical skills but also taught me the importance of persistence, curiosity, and a problem-solving mindset.

Thank you,

[sign. of the mentor]